

From Camera-Controlled Video to Spatial World Models

Technical foundations, an assessment of Atlas, and a development blueprint for Recomo

Version: 1.0 — Frozen baseline Date: 3 September 2026

Abstract

Camera-controlled video generation is moving from semantic motion prompts toward calibrated spatial modeling. The central technical progression is from frame-level camera pose conditions, to dense ray fields and epipolar constraints, to metric depth and explicit geometry, to persistent caches and 4D correspondence, and finally to models that jointly represent camera, video, depth, time, and world state. World Labs' Atlas is a strategically important signal for the last category, but its early public disclosure does not remove the need for explicit geometry, reproducible evaluation, cinematographic planning, or deterministic robot execution.

This white paper reviews the field, distinguishes several meanings of camera awareness, compares the major architectural families, and proposes a Recomo architecture built around a calibrated camera contract, persistent spatial state, geometry-conditioned generative rendering, independent verification, and a physical robot compiler. It also defines a staged path from a modular functional counterpart to an Atlas-like spatial foundation model.

1. The core conclusion

The field did not simply improve by translating “pan left” into a more accurate prompt. It changed the representation of the problem:

- Pose-conditioned generation: provide a trajectory to a pretrained video model.
- Dense geometric conditioning: convert camera calibration into spatially aligned rays and add multi-view constraints.
- Explicit geometry: reconstruct depth or point clouds, render known content at the requested pose, and use generation for uncertain or unseen regions.
- Native spatial modeling: treat camera, RGB, depth, time, and world context as jointly modeled variables.

For a real filming robot, the recommended system is

```
\[
\boxed{
\text{explicit camera contract}
+\text{persistent spatial representation}
+\text{camera-aware generative renderer}
+\text{cinematic planner}
+\text{independent verifier}
+\text{deterministic robot execution}
}.
\]
```

A video model should not be the sole authority for metric geometry or robot motion.

2. What camera-aware means

Capability	Real requirement	Does not prove
Semantic camera control	Understands pan, orbit, crane, truck, handheld	Numeric trajectory following
Extrinsic control	Accepts per-frame (R, t) or $(SE(3))$	Metric scale or optics
Calibrated ray control	Uses (K) and pose to define pixel rays	Scene depth or correct occlusion

Capability	Real requirement	Does not prove
Metric camera control	Translation is tied to real-world scale	Complete or accurate unseen geometry
Geometric consistency	Scene points reproject across views	Dynamic-world fidelity
Dynamic 4D consistency	Object motion and camera time remain synchronized	Long-horizon memory
Optical control	Focal length, distortion, focus, aperture, shutter	Physical executability
Persistent world modeling	Supports revisits and multiple paths through one scene	Guaranteed physical truth

The standard projective intrinsic matrix $\mathbf{K}$ must be distinguished from lens distortion, projection family, focus, aperture, shutter, and exposure. A dolly and a zoom may preserve apparent framing while producing different parallax.

There are also two separate tasks:

- trajectory planning: decide which camera movement is desired;
- trajectory-conditioned generation: render what the camera would see along a supplied path.

Most reviewed models solve the second task. Recomo must solve both and compile the result into physical motion.

3. Camera representation

For pixel $\tilde{p}=[u,v,1]^T$, camera-to-world rotation $\mathbf{R}_t$, centre $\mathbf{o}_t$, and intrinsics $\mathbf{K}_t$, the world ray is

$$\mathbf{d}_{tuv} = \frac{\mathbf{R}_t \mathbf{K}_t^{-1} \tilde{p}}{\|\mathbf{R}_t \mathbf{K}_t^{-1} \tilde{p}\|}.$$

A common Plücker representation is

$$\mathcal{P}_{tuv} = (\mathbf{o}_t \times \mathbf{d}_{tuv}, \mathbf{d}_{tuv}).$$

Dense rays align camera geometry with visual tokens. They still contain no scene depth: each token observes somewhere along a ray, not a known surface point. Large camera translations reveal content absent from the source, so models must hallucinate, retrieve prior observations, reconstruct explicit geometry, or combine these strategies.

4. Evolution of the field

4.1 MotionCtrl and pose-conditioned motion

MotionCtrl separated object motion and camera motion, showing that they should not be controlled as one undifferentiated “motion” variable. It established a practical route for adding camera trajectories to pretrained video generators.

4.2 CameraCtrl and dense ray conditioning

CameraCtrl constructed pixel-wise Plücker embeddings from camera calibration and injected multiscale camera features into temporal video layers. This made camera control spatially aligned and influenced many later systems.

4.3 CamCo, CamI2V, and epipolar constraints

CamCo and CamI2V introduced epipolar aggregation. Calibrated source and target cameras restrict a feature correspondence to an epipolar line, reducing the burden on learned attention and improving multi-view consistency. Their limitations also revealed the importance of dynamic-scene data and broad camera distributions.

4.4 VD3D and video diffusion transformers

VD3D moved camera conditioning from U-Net video models to transformer-based video diffusion through a ControlNet-like branch with spatiotemporal ray features.

4.5 4DiM and space-time generation

4DiM treated images, camera poses, and timestamps as central variables in a space-time diffusion model and aligned camera scale using metric depth. It anticipated a world-model-like formulation in which pose and time are controllable dimensions.

4.6 AC3D and when to inject camera control

AC3D analysed camera information inside a video diffusion transformer. It found that global camera motion is low-frequency, becomes established early in denoising, and can be recovered from intermediate representations. It therefore restricts camera conditioning to selected early layers and denoising stages.

The useful principle is:

Establish global camera transformation early; leave later capacity for appearance, local dynamics, identity, and detail.

AC3D-CogVideoX is an adapted backbone, not native spatial CogVideoX. CogVideoX's 3D VAE compresses space and time; it does not represent a metrically calibrated 3D world.

4.7 RealCam-I2V and metric scene alignment

RealCam-I2V predicts metric depth, backprojects the reference image, aligns pose scale, allows camera-path authoring inside the estimated scene, and uses geometric preview information to condition generation. It moves beyond relative trajectories, although accuracy remains dependent on monocular depth and calibration.

4.8 GEN3C and persistent 3D caches

GEN3C maintains a depth-derived cache of seed and generated frames, renders the cache along a requested trajectory, and conditions generation on those renderings. Known content is transported geometrically; the model generates disoccluded and uncertain content.

This is one of the most relevant open references for Recomo because it respects a clear division:

```
\[
\text{deterministic view transport}+\text{generative completion}.
\]
```

4.9 TrajectoryCrafter and Uni3C

TrajectoryCrafter applies rendered geometry to video redirection. Uni3C compares ray-based and point-cloud conditions and reinforces the practical value of explicit scene transport, while inheriting depth-estimation errors.

4.10 ReCamMaster, CameraCtrl II, and RealCam

These systems extend camera control to refilming dynamic videos, wider spatial exploration, and interactive/streaming operation. Dynamic refilming requires preserving world time—the exact timing of actors, objects, cloth, and lighting—while changing viewpoint.

4.11 Track2View and explicit 4D correspondence

Track2View conditions on projected 3D point tracks, explicitly specifying where source content should appear in target cameras. The progression becomes

```
\[
\text{pose condition}\rightarrow\text{rendered geometry}\rightarrow\text{4D correspondence}.
\]
```

4.12 Intrinsic and optics

AKiRa, UCPE, DeltaCam, and CameraAnything broaden the problem to focal length, distortion, pitch/roll, gravity, focus, aperture, exposure, aspect ratio, and multi-shot transitions. These variables are essential for a physical camera system and cannot be delegated to vague text prompts.

4.13 Rays as Pixels

Rays as Pixels represents cameras as image-like raxel tensors passing through the same spatiotemporal VAE as video. One model can estimate camera from video, generate video from a trajectory, or jointly generate both. This marks a transition from camera as a fixed condition to camera as a modeled modality.

5. Model-family comparison

Family	Camera input	Spatial mechanism	Strength	Main limitation
CogVideoX base	Text; image in I2V variants	Video latent prior	Strong open video backbone	No native calibrated camera contract
AC3D	Per-frame intrinsics/extrinsics converted to rays	Camera adapter/ControlNet branch	Efficient explicit trajectory control	No persistent metric scene state
RealCam-I2V	Image, calibration, depth-aligned trajectory	Estimated metric point cloud and preview	Scene-scale path authoring	Depends on monocular depth; ports differ from evaluated setup
GEN3C	Image/video and target trajectory	Rendered persistent 3D cache	Strong geometry/generation division	Cache and depth errors can accumulate
Track2View	Source/target cameras and 3D tracks	Explicit 4D correspondence	Precise content transport	Needs reliable geometry/tracks
Rays as Pixels	Joint camera and video latents	Shared spatiotemporal representation	Forward, inverse, and joint tasks	Data bias and limited long-term memory
Atlas	Posed images, depth, text, camera path	Shared native spatial context	Unified generation and reconstruction direction	Closed early access and incomplete public metrics

Paper headline numbers should not be directly ranked across different backbones, resolutions, datasets, pose estimators, and scale-alignment protocols.

6. Atlas assessment in v1.0

World Labs describes Atlas as a multimodal autoregressive diffusion transformer pretrained from scratch. Publicly described inputs include text, images, camera poses, and depth maps, with video represented as posed image sequences in a shared spatial context. Demonstrations include camera-controlled generation, reconstruction, depth, point clouds, Gaussian splats, and robotics-oriented simulation workflows.

This is not merely another camera adapter. Camera and geometry are presented as native parts of the foundation-model interface. That is a meaningful architectural signal.

However, the public camera comparison supplies Atlas with a native path while general video comparators receive textual camera descriptions. It supports the value of numeric camera input, but it is not a controlled comparison with specialist explicit-camera models. Public material also does not establish variable per-frame intrinsics, metric trajectory tolerance, lens distortion, loop closure, dynamic-world memory, latency, VRAM, model scale, or reproducible implementation.

Accordingly:

Atlas is a validation of direction, not yet a reproducible dependency or a complete engineering specification.

7. Open problems

7.1 Camera/object-motion entanglement

Strong camera conditioning can suppress actor motion; dynamic objects can corrupt camera estimation. A filming model must distinguish camera time from world time.

7.2 Metric integrity

Many evaluations align or normalize translation per clip. This measures path shape, not actual distance. Recomo requires a no-scale-alignment metric track.

7.3 Disocclusion

No model can recover unobserved truth from a single image. It can infer plausible content, but must label that content as generated rather than measured.

7.4 Long-horizon revisitation

Temporal smoothness over seconds does not guarantee a stable room after turning away and returning. Loop closure is a separate requirement.

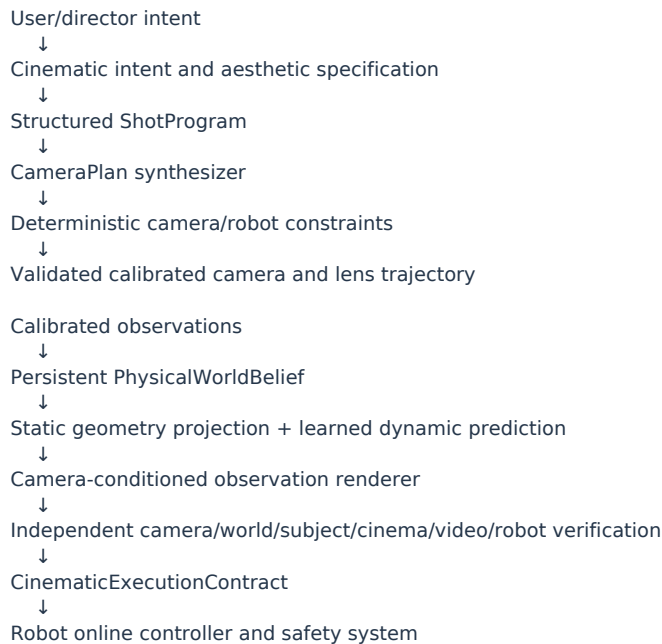
7.5 Gravity and non-level cameras

Datasets often lack a global gravity frame. Extreme pitch, roll, inverted views, and moving arm-mounted gimbal bases are underrepresented.

7.6 Physical executability

A visually plausible virtual trajectory may violate joint limits, collision, balance, stability, visibility, velocity, acceleration, or safety constraints.

8. Recommended Recomo architecture



The model can eventually merge logical functions, but the contracts and authority boundaries must remain exportable and testable.

Three state types

- PhysicalWorldBelief: geometry, posed-frame memory, entities, identity, robot/camera state, visibility, uncertainty, and provenance.
- CinematicContinuityState: shot phase, composition, motion/lens motif, palette, narrative arc.
- RendererLocalCache: model-specific latents and KV state; disposable and non-authoritative.

9. Canonical camera contract

$$\mathcal{C}(t) = \{T^W_C(t), K(t), \Pi(t), D(t), \text{focus}(t), \text{aperture}(t), \text{shutter}(t), \text{exposure}(t), \tau_t\}.$$

The contract must define units, coordinate handedness, axes, time base, shutter convention, calibration source, and uncertainty.

Physical composition is

$$\mathbb{T}^W_C(t) = \mathbb{T}^W_B(t) \mathbb{T}^B_A(q_t) \mathbb{T}^A_G(\theta_t) \mathbb{T}^G_C.$$

The model should consume the final camera observation trajectory. A separate deterministic compiler allocates it to chassis, arm/lift, wrist, and gimbal.

10. Evaluation design

Use three regimes:

- calibrated synthetic scenes;
- static repeated Recomo robot captures;
- dynamic multi-camera real scenes.

The trajectory suite should include pan/tilt/roll, dolly/truck/crane, orbit, look-at, compound motion, dolly zoom, large parallax, non-level gimbal base, yaw lock/unlock, loop closure, aggressive out-of-distribution paths, and independent camera/world time.

Report separate scorecards:

$$S_{\text{camera}}, \quad S_{\text{world}}, \quad S_{\text{cinema}}, \quad S_{\text{robot/system}}.$$

Rules:

- report metric translation without per-clip scale alignment;
- report scale-aligned trajectory shape separately;
- count camera-estimation failure as failure;
- score observed and generated regions separately;
- report peak VRAM, runtime, loading overhead, and failure rate;
- compare predicted previews with actual robot-captured results.

11. Building a counterpart

Functional counterpart

Integrate reconstruction, explicit 3D cache, target-camera rendering, RGB-D generative completion, 3D fusion, and independent verification. Demonstrate several paths through one scene.

Architectural counterpart

Train a joint text/RGB/depth/camera/time model supporting masked multimodal tasks, camera estimation, novel-view generation, RGB-D continuation, and eventually joint camera/video hypotheses.

Scale counterpart

Match open-domain visual quality, long duration, high resolution, context capacity, and product serving. This is a later foundation-scale objective, not the first deliverable.

A sensible model program—provisionally RSFM, Recomo Spatial Film Model—combines typed spatial elements, a joint camera/video representation, external geometry memory, and Recomo's cinematic/execution contracts.

12. Data and curriculum

Use exact synthetic geometry, Recomo-native metric captures, synchronized dynamic multi-camera capture, and carefully labelled pseudo-calibrated public/licensed video.

The most valuable proprietary tuple is

$$\{\text{initial observation}, \text{planned path}, \text{executed path}, \text{actual video}\}.$$

\]

Train in stages:

- camera and depth literacy;
- sparse-view reconstruction and generation;
- short RGB-D video with separate camera/world time;
- external-memory-conditioned continuation;
- joint camera/video hypotheses;
- explicit world and action outputs.

13. Deployment and roadmap

A 24 GB GPU is a deployment profile, not a foundation training platform. Use sequential residency: plan, reconstruct, unload, generate, unload, verify. Persist compact camera paths, RGB-D, point clouds, tracks, uncertainty, and selected latents rather than all large models simultaneously.

Suggested progression:

- 0–2 months: contracts, benchmark, baseline integration, data schema;
- 2–5 months: modular functional counterpart;
- 4–10 months: 1–3B RSFM research model;
- 9–18 months: autoregressive spatial context, longer RGB-D, refilming, dynamic time, and physical embodiment evaluation.

14. Final judgment

The correct direction is not to replace every spatial and robotic component with the newest video model. It is to make camera, geometry, images, depth, and time native, mutually connected variables while keeping measured state, physical constraints, and acceptance criteria explicit.

```
\[
\boxed{
\text{Atlas-like learned spatial intelligence}
+\text{explicit 3D state}
+\text{Recomo cinematic planning}
+\text{deterministic robot execution}
}.
\]
```

Atlas is a strong North Star. Recomo's defensible contribution is to make that spatial intelligence cinematographically purposeful, metrically testable, and physically executable.

Primary references

- World Labs, Atlas: <https://www.worldlabs.ai/blog/atlas>
- MotionCtrl: <https://arxiv.org/abs/2312.03641>
- CameraCtrl: <https://hehao13.github.io/projects-CameraCtrl/>
- CamCo: <https://arxiv.org/abs/2406.02509>
- CamI2V: <https://arxiv.org/abs/2410.15957>
- VD3D: <https://arxiv.org/abs/2407.12781>
- 4DiM: <https://arxiv.org/abs/2407.07860>
- AC3D: <https://arxiv.org/abs/2411.18673>
- CogVideoX: <https://arxiv.org/abs/2408.06072>

- RealCam-I2V: <https://arxiv.org/abs/2502.10059>
- GEN3C: <https://arxiv.org/abs/2503.03751>
- TrajectoryCrafter: <https://arxiv.org/abs/2503.05638>
- Uni3C: <https://arxiv.org/abs/2504.14899>
- ReCamMaster: <https://arxiv.org/abs/2503.11647>
- CameraCtrl II: <https://arxiv.org/abs/2503.10592>
- RealCam: <https://arxiv.org/abs/2605.06051>
- Track2View: <https://arxiv.org/abs/2606.15534>
- AKiRa: <https://arxiv.org/abs/2412.14158>
- UCPE: <https://github.com/chengzhag/UCPE>
- DeltaCam: <https://arxiv.org/abs/2605.25266>
- CameraAnything: <https://arxiv.org/abs/2607.24591>
- Rays as Pixels: <https://arxiv.org/abs/2604.09429>